

Formalizing Singular Homology in Lean 4: Homotopy Invariance, Subdivision, Mayer–Vietoris, and the Homology of Spheres

Jonathan Washburn
Recognition Physics Institute, Austin, Texas
washburn@recognitionphysics.org

July 18, 2026

Abstract

Mathlib, the mathematical library of the Lean 4 proof assistant, defines singular homology of topological spaces through the singular simplicial set and the alternating face map complex, and it proves essentially no theorems about the resulting functor. This paper reports a formalization, in Lean 4 over Mathlib, of the classical computational layer of singular homology with integer coefficients: the prism-operator proof of homotopy invariance, the long exact sequence of a pair, barycentric subdivision with its diameter estimate, the small-simplices and small-chains theorems, the Mayer–Vietoris long exact sequence with honest space-level induced maps, the homology of spheres (vanishing in the intermediate range and nonvanishing in the top degree), and the corollary that spheres of different dimensions are never homotopy equivalent. On top of this layer we formalize applications about complements of embedded circles and arcs in S^D for arbitrary topological embeddings, with no smoothness, PL, or tameness hypotheses: the complement of the unknot in S^3 has nonvanishing first homology, and the complement of an embedded arc in S^D has vanishing first homology in every dimension, a formalized case of Hatcher’s Theorem 2B.1. These combine into a client theorem: an embedded circle in S^D with homologically nontrivial complement exists exactly when $D = 3$. The development comprises about 8,400 lines across twelve modules, builds with zero `sorry`, and every capstone reports the axiom footprint `[propext, Classical.choice, Quot.sound]`.

1 Introduction

Singular homology is the first genuinely computable invariant that a student of algebraic topology meets, and its computational core is remarkably stable across textbooks: homotopy invariance by the prism operator, the long exact sequence of a pair, barycentric subdivision and the small-simplices argument, excision or Mayer–Vietoris, and then the homology of spheres with its classical corollaries (invariance of dimension, Brouwer’s fixed point theorem, separation theorems). Mathlib [3], the library of the Lean 4 proof assistant [4], recently acquired the definition: `AlgebraicTopology.singularHomologyFunctor` sends a topological space to the homology of the alternating face map complex of its singular simplicial set, with coefficients in a module. What Mathlib does not have is the computational layer. At the time of writing there is essentially no theorem in Mathlib about the values of this functor: no homotopy invariance, no exact sequences, no subdivision, no computation of $H_*(S^n)$.

This paper reports a development that builds that layer, working directly against Mathlib’s own definition of singular homology. No project-local surrogate stands in between. Every statement below about $H_n(X)$ is a statement about `singularHomologyFunctor` applied to the coefficient module $\mathbb{Z}$, so anything proved here composes with future Mathlib development without translation. The development goes far enough to support theorems of genuinely geometric content. Its final applications concern arbitrary topological embeddings of circles and arcs into spheres, with no smoothness, PL, or local flatness assumptions: the wild cases are included, and this is a strength of the homological method that the formalization preserves. As a single sentence of motivation beyond topology: the client theorem in Section 9 (nontrivial linking of embedded circles forces ambient dimension three) is consumed by a physics program, described in a companion paper [1], that derives spatial dimensionality from information-theoretic axioms; nothing in the present paper depends on that program.

1.1 Contributions

All results are formalized in Lean 4 over Mathlib, build with zero `sorry`, and are stated for singular homology with $\mathbb{Z}$ coefficients.

- **Homotopy invariance** (954 lines). The prism operator, its five face identities, the chain homotopy identity $\partial P + P\partial = g_{\#} - f_{\#}$, and the packaged consequences: homotopic maps induce equal maps on homology, and homotopy equivalences induce isomorphisms (Hatcher Theorem 2.10, Corollary 2.11).
- **The long exact sequence of a pair** (239 lines). For an injective continuous map $A \rightarrow X$, the short exact sequence of singular chain complexes and its homology long exact sequence, with the connecting homomorphism (Hatcher Theorem 2.16, for pairs presented by injections).
- **Barycentric subdivision** (1,587 lines). A reusable combinatorial theory of affine chains with cone recursions for the subdivision operator S and the homotopy T , the identities $\partial S = S\partial$ and $\partial T + T\partial = 1 - S$, transport to the singular chain complex, the $\frac{n}{n+1}$ diameter contraction, and the small-simplices theorem `exists_sdOpIter_small` (the geometric heart of Hatcher’s proof of excision, pp. 119–124).
- **Mayer–Vietoris** (1,791 lines). The small-chains subcomplex $C_*^{U,V}(X)$, the small-chains theorem (the inclusion induces an isomorphism on homology; Hatcher Proposition 2.21), the Mayer–Vietoris short exact sequence of chain complexes (which needs no openness), and the Mayer–Vietoris long exact sequence with induced maps that are honestly the space-level maps (openness enters only here).
- **Homology of spheres** (745 + 694 lines). The augmentation apparatus, $H_0(X) \cong \mathbb{Z}$ for path-connected X (as invertibility of the augmentation), vanishing of positive-degree homology for contractible spaces, the suspension isomorphism $H_{k+2}(S^{n+1}) \cong H_{k+1}(S^n)$ obtained from the Mayer–Vietoris connecting map, and the two dimension detectors: $H_k(S^n) = 0$ for $1 \leq k \neq n$ and $H_n(S^n) \neq 0$ for $n \geq 1$. Corollary: spheres of different dimensions are never homotopy equivalent.
- **Complement topology for wild embeddings** (308 + 249 + 629 + 870 lines). The complement of the flat unknot in S^3 has nonvanishing H_1 ; no embedded circle in S^0 or S^1 has homologically nontrivial complement; for $D \geq 2$, $D \neq 3$, the circle-complement question

Module	Content	Classical reference
<code>SingularPrism</code>	prism operator, homotopy invariance	Hatcher Thm. 2.10, Cor. 2.11
<code>SingularPair</code>	chain SES, LES of a pair	Hatcher Thm. 2.16
<code>SingularSubdivision</code>	barycentric subdivision, small simplices	Hatcher pp. 119–124
<code>SingularMayerVietoris</code>	small chains, MV sequence	Hatcher Prop. 2.21, §2.2
<code>SingularSphere</code>	H_0 , contractible vanishing, MV tools	Hatcher Prop. 2.6, Cor. 2.11
<code>SingularSphereGeometry</code>	sphere homology, dimension detectors	Hatcher Cor. 2.14
<code>UnknotComplementRetract</code>	unknot complement, $H_1 \neq 0$	retraction argument
<code>LinkingVanishingLowDim</code>	no detection in S^0, S^1	point-set topology
<code>LinkingVanishingHighDim</code>	reduction of circles to arcs	Hatcher §2.2, §2.B
<code>ArcComplementAcyclic</code>	arc complements H_1 -acyclic, all D	Hatcher Thm. 2B.1 (arc case)
<code>PublicSpine</code>	detector, binder, anti-cheat gate	—
<code>PublicSpineLinkingClosure</code>	<code>forces_D3</code> , <code>target_D3</code>	—

Table 1: The twelve modules and their classical anchors. Line counts by `wc` -1: 954, 239, 1587, 1791, 745, 694, 308, 249, 629, 870, 311, 42 (total 8,419).

reduces by Mayer–Vietoris to arcs; and embedded arcs in S^D have H_1 -acyclic complements in every dimension D , by a compact-support bisection argument (the arc case of Hatcher Theorem 2B.1, in degree one, for arbitrary topological embeddings).

- **The client theorem** (311 + 42 lines). The detector `DetectsNontrivialLinking` D (some embedded circle in S^D has complement with nonvanishing H_1) holds if and only if $D = 3$; packaged as a content-typed binder with an anti-cheat gate.

Table 1 maps modules to their classical references. All twelve modules live in the directory `IndisputableMonolith/Foundation/` of the source repository; a public mirror is available (Section 10).

1.2 Related work

Singular homology has been formalized before, in other systems and with other designs. Harrison developed singular homology in `HOL Light` [5], reaching the Eilenberg–Steenrod axioms and consequences including Brouwer’s fixed point theorem and Jordan–Brouwer separation; Paulson ported that development (about 11,400 lines) to Isabelle/HOL, where it ships with the distribution as `HOL-Homology` [6]. Those developments use a simple-type-theoretic encoding of spaces and hand-rolled free abelian groups. In the dependently typed world, the homotopy type theory community computes homotopy and cohomology groups synthetically: Brunerie’s $\pi_4(S^3) \simeq \mathbb{Z}/2$ [7], cellular cohomology in `HoTT` [8], and machine computation of the Brunerie number in cubical `Agda` [9]. Synthetic results speak about types with path structure; the singular homology of general topological spaces studied here is a different object, so the two do not directly overlap. In Lean, Murphy formalized the Brouwer fixed point theorem via a singular homology apparatus built for that purpose, before `Mathlib`’s `singularHomologyFunctor` existed [10]. `Mathlib` itself has extensive simplicial machinery (simplicial sets, Dold–Kan, simplicial homotopy), group and Galois cohomology, and de Rham cohomology in the geometry library, but the singular homology functor arrived bare. The present development differs from all of the above in one design decision: it proves theorems about `Mathlib`’s own functor, in `Mathlib`’s category language (`TopCat`, `ModuleCat` $\mathbb{Z}$, homological complexes), so that the layer can be upstreamed statement by statement (Section 11).

2 Background and the target object

2.1 Singular homology in Mathlib

Mathlib defines the singular simplicial set of a space X as the restricted Yoneda presheaf of the standard simplices: an n -simplex is a continuous map $\Delta^n \rightarrow X$, where Δ^n is `stdSimplex ℝ (Fin (n+1))`. Applying the free-module functor levelwise and taking the alternating face map complex produces the singular chain complex functor `singularChainComplexFunctor`; its homology is `singularHomologyFunctor`. The development fixes coefficients once and for all:

```
noncomputable abbrev SC (X : TopCat.{0}) : ChainComplex (ModuleCat.{0} ℤ) ℕ :=
  ((AlgebraicTopology.singularChainComplexFunctor (ModuleCat.{0} ℤ)).obj
    (ModuleCat.of ℤ ℤ)).obj X

noncomputable abbrev Hgrp (X : TopCat.{0}) (n : ℕ) : ModuleCat.{0} ℤ :=
  (SC X).homology n
```

Both are reducible abbreviations, so `Hgrp X n` is the value of Mathlib’s singular homology functor at X with $\mathbb{Z}$ coefficients; there is no project-local homology theory to reconcile. Throughout the paper $H_n(X)$ abbreviates this object and $C_n(X)$ the degree- n chain group.

A recurring working presentation: with $\mathbb{Z}$ coefficients the degree- n chain group is the coproduct $\coprod_{\sigma} \mathbb{Z}$ in `ModuleCat ℤ`, indexed by the singular n -simplices. The development names the index type `Idx X n`, the coproduct `Cgrp`, and the generator inclusion `gen X n σ`. Two normal-form lemmas (`gen_d` for the boundary, `gen_map` for functoriality) reduce every chain-level computation to signed sums over generators. This presentation is set up once in `SingularPrism` and reused by every later module.

2.2 Design choices

The development is classical. Choice and excluded middle are used freely, as is customary in Mathlib’s topology and category theory; there is no attempt at constructive content, and none of the results are claimed constructively. There is also no synthetic homotopy theory: spaces are topological spaces, homotopies are continuous maps $I \times X \rightarrow Y$, and the results are about the classical singular functor. Universe level is pinned to `TopCat.{0}`, which is where Mathlib’s spheres live.

Every capstone theorem in the development reports the axiom footprint

`[proptext, Classical.choice, Quot.sound]`

under `#print axioms`: the three standard Lean axioms, and nothing else. No result depends on any project-local axiom.

3 Homotopy invariance: the prism operator

The module `SingularPrism` (954 lines) formalizes the textbook prism-operator proof of homotopy invariance, following Hatcher’s proof of Theorem 2.10 but reorganized so that the combinatorial cancellation is a standalone abstract lemma.

3.1 Topological prisms and face identities

The prism decomposition writes $\Delta^n \times I$ as a union of $(n + 1)$ simplices of dimension $n + 1$. Rather than triangulate literally, the formalization works with the $(n + 1)$ affine maps $\text{prism } i : \Delta^{n+1} \rightarrow \Delta^n \times I$ for $i : \text{Fin } (n + 1)$, and proves the five face identities that the cancellation argument needs: the composites of $\text{prism } i$ with the topological face inclusions $\text{face } j : \Delta^n \rightarrow \Delta^{n+1}$ satisfy $\text{prism_comp_face_top}$, $\text{prism_comp_face_bot}$, $\text{prism_comp_face_cancel}$, $\text{prism_comp_face_of_le}$, and $\text{prism_comp_face_of_gt}$. These are proved by direct computation with the affine coordinates of `stdSimplex`.

Given a homotopy $H : C(I \times X, Y)$ and a singular n -simplex σ of X , each prism map produces a singular $(n + 1)$ -simplex of Y (`prismSimplex`), and the prism operator is the signed sum over the decomposition, extended to chains by the universal property of the coproduct:

```
noncomputable def prismOp {X Y : TopCat.{0}} (H : C(I × X, Y)) (n : ℕ)
  (prisms : Fin (n + 1) →
    C(stdSimplex ℝ (Fin (n + 2)), stdSimplex ℝ (Fin (n + 1)) × I)) :
  Cgrp X n → Cgrp Y (n + 1) :=
  Sigma.desc (Pgen H n prisms)
```

3.2 The chain homotopy

The heart of the proof is an abstract double-sum cancellation (`prism_sum_cancellation`): for coefficients in any abelian group, the alternating double sum over faces of prisms collapses to the top and bottom faces once the diagonal terms cancel in pairs. The five face identities feed exactly this lemma, transported through the generator normal forms. The result is the chain homotopy identity, stated in positive degrees and in degree zero:

Theorem 3.1 (prism chain homotopy; `prism_chain_homotopy_succ, ..._zero`). *Let $F_0, F_1 : X \rightarrow Y$ be continuous and H a homotopy from F_0 to F_1 . Writing P for the prism operator of H and $f_\#$ for the induced chain map, on degree $n + 1$ chains*

$$\partial \circ P + P \circ \partial = (F_1)_\# - (F_0)_\#, \quad \text{and on degree 0, } \partial \circ P = (F_1)_\# - (F_0)_\#.$$

```
lemma prism_chain_homotopy_succ (Ho : ContinuousMap.Homotopy F_0.hom F_1.hom) (n : ℕ) :
  bnd X n >> prismOp Ho.toContinuousMap n prism +
  prismOp Ho.toContinuousMap (n + 1) prism >> bnd Y (n + 1) =
  chainMap F_1 (n + 1) - chainMap F_0 (n + 1)
```

Packaging the two identities as a Mathlib Homotopy of chain complex morphisms yields the module's exports.

Theorem 3.2 (homotopy invariance; `homotopic_maps_induce_same_homology`). *Homotopic continuous maps $f, g : X \rightarrow Y$ induce equal maps $H_n(X) \rightarrow H_n(Y)$ for every n , where H_n is Mathlib's singularHomologyFunctor with $\mathbb{Z}$ coefficients.*

Theorem 3.3 (homotopy equivalence; `homotopyEquiv_homology_iso`). *A homotopy equivalence $X \simeq Y$ induces an isomorphism $H_n(X) \cong H_n(Y)$ for every n , and the map induced by the forward direction is an isomorphism in ModuleCat $\mathbb{Z}$ (`isIso_homology_map_of_homotopyEquiv`).*

These are Hatcher Theorem 2.10 and Corollary 2.11. Every downstream module consumes homotopy invariance only through Theorem 3.3.

4 Relative homology and the long exact sequence of a pair

The module `SingularPair` (239 lines) is short because it delegates the homological algebra to Mathlib’s `ShortComplex.ShortExact` API. The geometric input is one lemma: for an *injective* continuous map $f : A \rightarrow X$, the induced chain map is a split monomorphism in every degree (`chainMap_mono`, `sChainMap_mono`). Injectivity suffices, no embedding hypothesis is needed, because a singular simplex of X that factors through the image factors uniquely through A at the level of index types, and the retraction `genRetract` is defined on generators by that factorization.

The relative complex is a cokernel, and the pair sequence is short exact:

```
noncomputable def relSC (f : A → X) : ChainComplex (ModuleCat.{0} ℤ) ℕ :=
  cokernel (sChainMap f)

lemma pairSES_shortExact (f : A → X) (hf : Function.Injective f.hom) :
  (pairSES f).ShortExact
```

Theorem 4.1 (LES of a pair; `pairδ`, `pair_les_exact1`, `pair_les_exact2`, `pair_les_exact3`). For injective continuous $f : A \rightarrow X$ there is a connecting homomorphism $\partial : H_{n+1}(X, A) \rightarrow H_n(A)$, and the sequence

$$\cdots \rightarrow H_{n+1}(X) \rightarrow H_{n+1}(X, A) \xrightarrow{\partial} H_n(A) \rightarrow H_n(X) \rightarrow H_n(X, A) \rightarrow \cdots$$

is exact at every position, in all degrees including zero.

This is Hatcher Theorem 2.16 for pairs presented by injections. A sanity theorem closes the module: $H_n(X, X) = 0$ (`relative_homology_id_isZero`). The same split-mono lemma is reused in Section 6 for the Mayer–Vietoris columns.

5 Barycentric subdivision and small simplices

The module `SingularSubdivision` (1,587 lines) is the technical center of the development. The excision-grade input that Mayer–Vietoris needs is: any singular chain can be subdivided, without changing its homology class, until every simplex lands in U or in V . Textbooks prove this with about four pages of formulas (Hatcher pp. 119–124); the formalization’s main design decision is to do all the algebra on a combinatorial layer where the formulas become structural recursions.

5.1 The affine chain layer

An affine n -simplex in a type α is just a vertex tuple, and an affine chain is a finitely supported $\mathbb{Z}$ -linear combination of tuples:

```
abbrev AC (α : Type) (n : ℕ) : Type := (Fin (n + 1) → α) →₀ ℤ
```

Faces are tuple reindexings along `Fin.succAbove`; the cone with apex b is `Fin.cons b`; the boundary `abnd` is the alternating sum of faces; the augmentation `eps` sums coefficients. No topology appears at this layer, and, importantly, no geometry either: the barycenter enters as an *abstract apex function* `bary : (Fin (m + 1) → α) → α`, a parameter of the constructions. The subdivision operator and its chain homotopy are the classical cone recursions

$$S\lambda = b_\lambda \cdot S(\partial\lambda), \quad T\lambda = b_\lambda \cdot (\lambda - T(\partial\lambda)),$$

implemented as `asub` and `atee` by structural recursion on degree. All chain identities hold at this level for *any* apex function (degree zero needs only `bary 0 w = w 0`): the cone boundary identity, $\partial\partial = 0$, the chain-map property $\partial S = S\partial$ (`abnd_comp_asub`), and the homotopy identity:

Theorem 5.1 (affine chain homotopy; `abnd_comp_atee`). *For every apex function `bary` and every degree n , $\partial \circ T + T \circ \partial = \text{id} - S$ on affine $(n + 1)$ -chains, where $S = \text{asub}$ and $T = \text{atee}$.*

```
theorem abnd_comp_atee (bary : ∀ {m : ℕ}, (Fin (m + 1) → α) → α) :
  ∀ n, (abnd (n + 1)).comp (atee bary (n + 1)) + (atee bary n).comp (abnd n)
    = LinearMap.id - asub bary (n + 1)
```

The iterates S^k come with a telescoped homotopy (`ateeIter`, `abnd_comp_ateeIter`): $\partial T_k + T_k \partial = 1 - S^k$. Proving these identities by structural recursion over a two-line grammar of generators is where the formalization gains the most over a literal transcription of the textbook, which manipulates the same content through nested summation indices.

5.2 Geometry: the honest barycenter and the diameter estimate

Stage 5b instantiates the abstract layer on $\alpha = \text{stdSimplex } \mathbb{R} \text{ (Fin } (d + 1))$: the realization `affineMap` of a vertex tuple as a continuous affine map, and the honest barycenter `sbary`, with its equivariance `sbary_affineMap` under affine maps. Equivariance is what lets the abstract identities transport: pushforward `amap` along a barycenter-preserving map commutes with `asub` and `atee`.

The quantitative content is Stage 6. The lemma `asub_support_bound` tracks supports through the cone recursion: each piece of a subdivided simplex has vertex spread at most $\frac{n}{n+1}$ times the original, by the convex-hull maximum principle. Iterating (`asubIter_support_bound`) gives the geometric decay $\left(\frac{n}{n+1}\right)^k$, and therefore:

Theorem 5.2 (uniform smallness; `exists_asubIter_small`). *For every $\varepsilon > 0$ and every vertex tuple w on the standard simplex, there is an iterate k such that every tuple in the support of $S^k(w)$ has all pairwise vertex distances below ε .*

5.3 Transport and the small-simplices theorem

Stage 5c transports the operators to the singular chain complex. On a generator σ , the singular subdivision `sdOp` is the pushforward along σ of the subdivision of the identity tuple; the homotopy `tOp` is transported the same way. The chain-map property (`sdOp_comp_bnd`), the homotopy identities (`tOp_chain_homotopy_succ`, `tOp_chain_homotopy_zero`), naturality in the space (`sdOp_natural`, `tOp_natural`), and the iterated versions (`sdOpIter`, `tOpIter`, `tOpIter_chain_homotopy_succ`) all descend from the affine layer through the generator normal forms.

The capstone combines Theorem 5.2 with the Lebesgue number lemma on the compact metric space Δ^n :

Theorem 5.3 (small-simplices theorem; `exists_sdOpIter_small`). *Let U, V be open with $U \cup V = X$ and let s be a singular n -simplex of X . Then there is an iterate k such that every piece of `sdOpIter` X n k applied to the generator of s has range contained in U or contained in V .*

```
theorem exists_sdOpIter_small {n : ℕ} (U V : Set X) (hU : IsOpen U) (hV : IsOpen V)
  (hUV : U ∪ V = Set.univ) (s : Idx X n) :
  ∃ k, ∀ u ∈ ((asubIter (baryFn n) k n) (asimplex (idTuple n))).support,
    Set.range ↑(simplexEquiv X n (pushSimplex (simplexEquiv X n s) u)) ⊆ U ∨
    Set.range ↑(simplexEquiv X n (pushSimplex (simplexEquiv X n s) u)) ⊆ V
```

Together with the homotopy witness $1 - S^k = \partial T_k + T_k \partial$, this is exactly the input the Mayer–Vietoris layer consumes: subdivision reaches the small subcomplex, and the reached chain is homologous to the original.

6 Mayer–Vietoris

The module `SingularMayerVietoris` (1,791 lines) assembles the subdivision layer into the Mayer–Vietoris long exact sequence. Fix $U, V \subseteq X$ (arbitrary subsets at first).

6.1 The small-chains subcomplex and the small-chains theorem

A singular simplex is `Small U V` if its range lies in U or in V ; the small chains $C_n^{U,V}(X)$ are the span of the small generators, assembled into a subcomplex `SSC U V` with inclusion `smallU`. The main theorem of the first half is Hatcher’s Proposition 2.21 specialized to two-set covers:

Theorem 6.1 (small-chains theorem; `smallU_isIso_homologyMap`, `smallChainsHomologyIso`). *For open U, V with $U \cup V = X$, the inclusion $C_*^{U,V}(X) \rightarrow C_*(X)$ induces an isomorphism on homology in every degree.*

The proof is elementwise, through a concrete surjectivity-and-injectivity criterion for homology maps of complexes of $\mathbb{Z}$ -modules (`isIso_homologyMap_of_elementwise`): every cycle of $C_*(X)$ subdivides into the subcomplex (Theorem 5.3, with the iterate k uniformized over the finitely many generators of the chain by taking the maximum), and the homotopy operator T_k , which preserves the subcomplex by naturality, witnesses that nothing is lost. Hatcher proves more (a chain homotopy inverse); the homology-level isomorphism is what Mayer–Vietoris needs, and it is what the formalization exports.

6.2 The Mayer–Vietoris sequence

The short exact sequence lives at the chain level and needs no openness at all:

```

noncomputable def mvα :
  SC (TopCat.of (U ∩ V : Set X)) → SC (TopCat.of U) ⊕ SC (TopCat.of V) :=
  biprod.lift (sChainMap (mvInclU U V)) (-(sChainMap (mvInclV U V)))

noncomputable def mvβ :
  SC (TopCat.of U) ⊕ SC (TopCat.of V) → SSC U V :=
  biprod.desc (smallU U V) (smallV U V)

theorem mvSES_shortExact : (mvSES U V).ShortExact

```

The observation that $0 \rightarrow C_*(U \cap V) \rightarrow C_*(U) \oplus C_*(V) \rightarrow C_*^{U,V}(X) \rightarrow 0$ is exact for *arbitrary* subsets, openness entering only when the small subcomplex is compared to the full complex, is standard but easy to blur in informal treatments; the formal statement keeps the hypotheses honest. Degree-wise exactness in the middle (`mv_middle_exact`) is a direct combinatorial argument on coproduct coordinates: a pair of chains agreeing after inclusion must come from the intersection, generator by generator.

Combining with Theorem 6.1 and the homology LES of a short exact sequence of complexes gives the space-level sequence. The maps matter: `mvPair` is (induced by) the two inclusions $U \cap V \rightarrow U$, $U \cap V \rightarrow V$ with a sign, `mvSum` is the sum of the maps induced by $U \rightarrow X$, $V \rightarrow X$, and the

connecting map `mvδ` is the LES connecting map conjugated by the small-chains isomorphism. All three are honest space-level maps of Mathlib’s homology functor, so downstream users can compose them with arbitrary induced maps.

Theorem 6.2 (Mayer–Vietoris LES; `mvδ`, `mv_exact1`, `mv_exact2`, `mv_exact3`, `mvSum_epi_zero`). *For open U, V with $U \cup V = X$, the sequence*

$$\cdots \rightarrow H_{n+1}(X) \xrightarrow{\partial} H_n(U \cap V) \rightarrow H_n(U) \oplus H_n(V) \rightarrow H_n(X) \rightarrow \cdots$$

is exact at every position; in degree zero the final map is an epimorphism.

Remark 6.3 (engineering: the `ModuleCat ℤ` instance diamond). One engineering lesson deserves a highlighted place, because it will affect anyone who does elementwise arguments in complexes of `ModuleCat ℤ`. The elements of a chain group carry a $\mathbb{Z}$ -module structure through two unification paths: the categorical instance of `ModuleCat ℤ`, and the generic chain `AddCommGroup.toIntModule` (with its `SubNegMonoid.toZSMul` scalar action). The two are propositionally but not definitionally equal, and Lean’s elaborator will happily pick different paths in different subterms of one goal, producing goals that look identical and refuse to close. Sixteen hundred lines of `Finsupp` and span manipulation became feasible only after pinning the priorities locally:

```
attribute [local instance 0] AddCommGroup.toIntModule
attribute [local instance 0] SubNegMonoid.toZSMul
```

Demoting the two generic instances to the lowest priority makes elaboration choose the categorical instance uniformly. The same two lines are repeated in every downstream module that computes with elements (`SingularSphere`, `SingularSphereGeometry`, `LinkingVanishingHighDim`, `ArcComplementAcyclic`).

7 The homology of spheres

Two modules compute sphere homology: `SingularSphere` (745 lines) builds the abstract tools, `SingularSphereGeometry` (694 lines) supplies the geometry. The sphere model is `Sph n = Metric.sphere (0 : ℝn+1) 1` as an object of `TopCat`.

7.1 Degree zero and contractible spaces

The augmentation `augTo` sends every 0-simplex to $1 \in \mathbb{Z}$; on homology it gives `augH` : $H_0(X) \rightarrow \mathbb{Z}$, and a choice of point gives a section `ptH`.

Theorem 7.1 (H_0 of a path-connected space; `isIso_augH_of_pathConnected`). *For a path-connected space X , the augmentation $H_0(X) \rightarrow \mathbb{Z}$ is an isomorphism in `ModuleCat ℤ`.*

This is Hatcher Proposition 2.6, stated as invertibility of a specific map so that later arguments can chase the augmentation through induced maps; a bare isomorphism statement would not support that chase. Positive degrees of a point vanish by inspection of the complex (`isZero_homology_of_totallyDisconnected` handles any totally disconnected space), and homotopy invariance transports this:

Theorem 7.2 (contractible vanishing; `isZero_homology_of_contractible`). *For contractible X and $m \neq 0$, the object $H_m(X)$ is a zero object.*

The module also proves the abstract Mayer–Vietoris induction tools: if U, V are open, cover X , and have contractible-like vanishing, the connecting map is an isomorphism in the relevant range (`isIso_mvδ`, `isIso_mvδ_of_contractible`), plus the degree-one edge cases (`isZero_h1_of_contractible`, `mono_mvPair_zero`).

7.2 Geometry: covers, stereographic projection, and suspension

The geometric module covers S^{n+1} by the complements $U = S^{n+1} \setminus \{\text{south}\}$ and $V = S^{n+1} \setminus \{\text{north}\}$. Each is contractible: the formalization proves `contractibleSpace_compl_singleton_sphere` through Mathlib’s stereographic projection. The intersection deformation-retracts to the equator: `interHomeoPunctured` identifies $U \cap V$ with a punctured hyperplane, polar coordinates `puncturedPolar` split it as $S^n \times (0, \infty)$ -like data, and `hequivProdContractible` collapses the contractible factor, producing the homotopy equivalence `interHomotopyEquiv` between $U \cap V$ and S^n . Feeding these into the abstract Mayer–Vietoris induction yields the suspension isomorphism as a composite of the connecting map and the equator identification:

```
noncomputable def suspensionIso (n k : ℕ) :
  Hgrp (Sph (n + 1)) (k + 2) ≅ Hgrp (Sph n) (k + 1)
```

7.3 The dimension detectors

Induction on n with base cases S^0 (two points; homology vanishes in positive degrees by total disconnectedness) and S^1 (where the degree-one case is handled by exactness with both cover pieces contractible) gives the two exports that all applications consume:

Theorem 7.3 (`vanishing`; `sphere_homology_vanish`). *For all n and all k with $1 \leq k$ and $k \neq n$, the object $H_k(S^n)$ is a zero object.*

Theorem 7.4 (`nonvanishing`; `h1_s1_ne_zero`, `sphere_top_ne_zero`). *$H_1(S^1)$ is not a zero object, and for every $n \geq 1$, $H_n(S^n)$ is not a zero object.*

A remark on honesty of scope: the development exports `vanishing` and `nonvanishing`, and does not export an isomorphism $H_n(S^n) \cong \mathbb{Z}$ for general n . Nonvanishing is proved by chasing a nonzero connecting map down the suspension ladder to $H_1(S^1) \neq 0$; the exactness bookkeeping for that chase does not require identifying the group. For $n = 1$ specifically, a separate, earlier module of the same repository (`CircleWindingChain`, 7,753 lines, predating this campaign) proves the isomorphism $H_1(S^1) \cong \mathbb{Z}$ as `circleH1ZIsoInt_holds`, by an explicit winding-chain computation; Section 8 uses it. Deriving $H_n(S^n) \cong \mathbb{Z}$ for all n from `suspensionIso` and the $n = 1$ case is a short induction left for the upstreaming pass (Section 11).

The classical corollary follows by comparing detectors across a homotopy equivalence:

Theorem 7.5 (`spheres detect dimension`; `spheres_not_homotopyEquivalent`). *For $m \neq n$ there is no homotopy equivalence between S^m and S^n ; equivalently, a homotopy equivalence between S^m and S^n forces $m = n$ (`sphere_dim_eq_of_homotopyEquiv`).*

```
theorem spheres_not_homotopyEquivalent {m n : ℕ} (hmn : m ≠ n) :
  IsEmpty (ContinuousMap.HomotopyEquiv ↑(Sph m) ↑(Sph n))
```

8 Applications: complements of wild circles and arcs

The application layer studies, for each ambient dimension D , whether some embedded circle in S^D has a complement with nonvanishing first homology. Everything in this section works with arbitrary `Topology.IsEmbedding` maps. No smoothness, PL structure, local flatness, or tameness is assumed anywhere, so wild embeddings (Antoine, Fox–Artin, and worse) are within scope of every statement. This generality costs nothing extra, because the method is homological throughout; it is a concrete payoff of formalizing the singular theory itself, which a tame surrogate would not deliver. The scope is deliberately narrow in a different direction: the development proves a corridor of Alexander-type complement control (complements of arcs in degree one, plus the specific circle statements below), and does not prove, or claim, full Alexander duality.

8.1 Dimension three: the unknot has nontrivial complement

The module `UnknotComplementRetract` (308 lines) proves existence at $D = 3$ with a self-contained retraction argument. The flat unknot is $z \mapsto (z, 0, 0)$; the core circle of its complement is $w \mapsto (0, 0, w)$; and the retraction normalizes the last two coordinates:

Theorem 8.1 (unknot complement; `unknotComplementH1_ne_zero`). *Assume $H_1(S^1) \cong \mathbb{Z}$. Then the first singular homology of the complement of the flat unknot in S^3 is not a zero object.*

```
theorem unknotComplementH1_ne_zero
  (h1 : Nonempty (((AlgebraicTopology.singularHomologyFunctor (ModuleCat ℤ) 1).obj
    (ModuleCat.of ℤ ℤ)).obj (TopCat.sphere.{0} 1)) ≅ ModuleCat.of ℤ ℤ)) :
  ¬ CategoryTheory.Limits.IsZero
  (((AlgebraicTopology.singularHomologyFunctor (ModuleCat ℤ) 1).obj
    (ModuleCat.of ℤ ℤ)).obj
    (TopCat.of {x : TopCat.sphere.{0} 3 // x ∉ Set.range unknot})))
```

Functoriality alone finishes: `retract_core` shows the composite $S^1 \rightarrow \text{complement} \rightarrow S^1$ is the identity, so $H_1(S^1) \cong \mathbb{Z}$ is a retract of $H_1(\text{complement})$, which therefore cannot be zero. The hypothesis is discharged by `circleH1ZIsoInt_holds` (Section 7) when the statement is consumed in Section 9. Notably, this existence half uses no homotopy invariance, no subdivision, and no Mayer–Vietoris: it is a 308-line functoriality argument available even before the computational layer.

8.2 Low dimensions: no room for linking

The module `LinkingVanishingLowDim` (249 lines) disposes of $D = 0$ and $D = 1$ by point-set topology.

Theorem 8.2 (low-dimensional vanishing; `not_detects_zero`, `not_detects_one`). *No embedded circle in S^0 or in S^1 has a complement with nonvanishing H_1 .*

For $D = 0$ there is no embedded circle at all: S^0 is a two-point discrete space and homology of any subspace is concentrated in degree zero, by `isZero_homology_of_totallyDisconnected` again. For $D = 1$, a continuous injection from S^1 to itself is surjective, by `continuous_injective_circle_self_surjective`. That proof runs through the intermediate value theorem, via `no_continuous_injective_circle_to_real`. So the complement of an embedded circle in S^1 is empty and its homology vanishes.

8.3 High dimensions: reduction to arcs

The module `LinkingVanishingHighDim` (629 lines) handles all $D \geq 2$, $D \neq 3$ at once, conditionally on one clean frontier hypothesis:

```
def ArcComplementsAcyclic (D : ℕ) : Prop :=
  ∀ a : C(unitInterval, ↑(Sph D)), Topology.IsEmbedding a →
    CategoryTheory.Limits.IsZero
      (Hgrp (TopCat.of {y : ↑(Sph D) // y ∉ Set.range a}) 1)
```

The reduction is Mayer–Vietoris. Split the embedded circle into two closed semicircular arcs meeting in two points. The complements of the two arcs form an open cover of the two-point complement $S^D \setminus \{p, q\}$, their intersection is the circle complement, and the sequence

$$H_2(S^D \setminus \{p, q\}) \rightarrow H_1(\text{circle complement}) \rightarrow H_1(\text{arc complement}) \oplus H_1(\text{arc complement})$$

traps the middle term. The two outer terms vanish: the arcs by hypothesis, and the left term because $S^D \setminus \{p, q\}$ is homotopy equivalent to S^{D-1} (`twoPointComplHEquiv`, built from stereographic projection, a translation homeomorphism, and the polar splitting), whose H_2 vanishes for $D \neq 3$ by Theorem 7.3 (`isZero_h2_twoPointCompl`). A pleasant surprise fell out of the uniform treatment: the case $D = 2$ needs no Jordan–Schoenflies theorem and no separation theory whatsoever, because the argument only ever consults $H_2(S^1)$, which vanishes. The classical intuition that dimension two is special (complements of circles in S^2 disconnect) is real but irrelevant to this statement.

Theorem 8.3 (high-dimensional reduction; `forces_D3_of_arcAcyclic`). *Fix D with $1 \leq D$ and $D \neq 3$, and assume `ArcComplementsAcyclic D`. Then every embedded circle in S^D has H_1 -acyclic complement (`isZero_h1_complement_of_embedding`). Consequently, granting the arc hypothesis for all $D \geq 2$, $D \neq 3$: if some embedded circle in S^D has homologically nontrivial complement, then $D = 3$.*

8.4 Arc complements are acyclic in every dimension

The module `ArcComplementAcyclic` (870 lines) discharges the hypothesis unconditionally, in all dimensions including $D = 3$. This is the arc case of Hatcher’s Theorem 2B.1, in degree one, for arbitrary topological embeddings.

Theorem 8.4 (arc complements; `arcComplementsAcyclic`). *For every D and every topological embedding $a : [0, 1] \rightarrow S^D$, the object $H_1(S^D \setminus a([0, 1]))$ is a zero object.*

The proof is the classical bisection. Suppose some 1-cycle z in the complement of the arc is not a boundary. The module first builds an elementwise toolkit for homology classes of complexes of $\mathbb{Z}$ -modules (`classOf`, `classOf_eq_zero_iff`, `classOf_natural`) and its topological wrappers (`cls`, `exists_nonbounding`, `bounds_of_isZero`). Splitting the arc at its midpoint, the complements of the two half-arcs cover the complement of their common point; Mayer–Vietoris exactness at $H_1(U \cap V)$ (`bounds_of_mv`), with H_2 and H_1 of the punctured sphere vanishing by Section 7 (the punctured sphere is contractible), shows z must fail to bound in one of the two half-arc complements (`bounds_of_halves`). Iterating produces a nested sequence of parameter intervals (`badSeq`) converging to a single point t^* . The complement of the point $a(t^*)$ is contractible, so z bounds there; a bounding chain is a finite object, its compact support avoids $a(J)$ for a small enough interval $J \ni t^*$ around the limit (`exists_chain_lift`, the compact-support lifting lemma), and that contradicts badness of the interval J . The limit step is where formalization pressure was

highest: “a chain lives in a finite subcomplex” is a triviality on paper and a genuine lemma (`cPush`, `cLift`, `exists_chain_lift`) about filtered behavior of `Finsupp`-supported chains here.

Theorem 8.4 states degree one only, which is exactly what Theorem 8.3 consumes. The full Theorem 2B.1 (all reduced degrees, and disk complements in all codimensions) remains open in this development; the bisection scaffolding is degree-generic and should extend.

9 The client theorem

The module `PublicSpine` (311 lines) states the campaign-facing definitions. The 42-line module `PublicSpineLinkingClosure` closes them. The detector is content-typed: it is defined on Mathlib’s homology object itself, so no proxy re-statement can satisfy it.

```
def DetectsNontrivialLinking (D : ℕ) : Prop :=
  ∃ f : C(TopCat.sphere.{0} 1, TopCat.sphere.{0} D),
    Topology.IsEmbedding f ∧
    ¬ CategoryTheory.Limits.IsZero (linkingComplementH1 D f)
```

Theorem 9.1 (the client theorem, in two halves; `detectsNontrivialLinking_three`, `forces_D3`). *Existence at three: `DetectsNontrivialLinking 3` holds (`detectsNontrivialLinking_three`), which is Theorem 8.1 with the hypothesis discharged by `circleH1ZIsoInt_holds`. Uniqueness: for every D , `DetectsNontrivialLinking D` implies $D = 3$ (`forces_D3`), assembled from Theorems 8.2, 8.3 and 8.4. Together the two halves give `DetectsNontrivialLinking D` if and only if $D = 3$. No single Lean theorem states that biconditional; it is the conjunction of the two named declarations.*

The two halves are packaged for the consuming program by a separate bridge declaration `target_D3`, described below.

```
theorem forces_D3 :
  ∀ D, PublicSpine.DetectsNontrivialLinking D → D = 3 :=
  PublicSpineLinkingAssembly.forces_D3_of_arcAcyclic
  (fun D _ => ArcComplementAcyclic.arcComplementsAcyclic D)
```

One paragraph on the packaging, with details in the companion paper [1]. The consuming program requires the result as a binder, `AlexanderLinkingBridge`, a structure whose fields are the $H_1(S^1) \cong \mathbb{Z}$ isomorphism, the detection statement at $D = 3$, and the uniqueness implication; `target_D3` inhabits it unconditionally. Because autonomous provers contributed to this campaign, the repository enforces an anti-cheat discipline. A CI gate script inspects the source file `PublicSpine.lean`: it checks that the target stays a `def`, rejects a `theorem target_D3` declaration in that file, and flags known drift patterns in the guidepost `Skeleton` module. The gate is a source-text check on those files and does not scan the whole repository. The binder fields are content-typed against Mathlib’s own objects (a homology object of a concrete subspace of a concrete sphere), so the target cannot be “satisfied” by redefining the statement, weakening the detector, or introducing a parallel homology theory. The conditional assembly (`PublicSpineLinkingAssembly`, 49 lines) was kept as a separate module so that the exact logical distance between the Mayer–Vietoris reduction and the unconditional closure stayed visible while the arc lemma was still open.

10 Artifact and reproducibility

10.1 Repository and build

The development lives in the `IndisputableMonolith` Lean library; the twelve modules of this paper are under `IndisputableMonolith/Foundation/`. A public mirror of the campaign modules is available at

<https://github.com/jonwashburn/shape-of-logic>, commit
3606692 (80a2a84f28b67f2a0edd99756fba7c9fc).

The library builds with Lake against a pinned Mathlib. Each module builds as a single target, for example:

```
$ lake build IndisputableMonolith.Foundation.SingularMayerVietoris
$ lake build IndisputableMonolith.Foundation.PublicSpineLinkingClosure
```

All twelve modules build green with zero `sorry`. The axiom audit on the capstones (`arcComplementsAcyclic`, `forces_D3`, `target_D3`, and the exports of every section above) reports exactly:

```
#print axioms IndisputableMonolith.Foundation.PublicSpineLinkingClosure.target_D3
-- 'IndisputableMonolith.Foundation.PublicSpineLinkingClosure.target_D3'
-- depends on axioms: [propext, Classical.choice, Quot.sound]
```

10.2 Theorem index

Table 2 maps every numbered statement of this paper to its Lean declaration and module.

11 Conclusion and upstreaming

The classical computational layer of singular homology now exists in Lean 4 over Mathlib’s own definitions: homotopy invariance, the LES of a pair, barycentric subdivision with its metric estimate, small chains, Mayer–Vietoris, sphere homology detectors, and a set of complement theorems strong enough to characterize an ambient dimension. Three formalization lessons stand out. First, doing the subdivision algebra on an abstract affine-chain layer with a parametric apex turned the hardest textbook pages into structural recursions, and the geometry entered exactly once, through the barycenter’s equivariance and the diameter bound. Second, elementwise arguments in `ModuleCat  $\mathbb{Z}$` complexes are entirely feasible at this scale, but only after the instance diamond of Remark 6.3 is pinned. Third, hypotheses stayed more honest than the textbook default: injectivity (not embedding) drives the pair sequence, openness is absent from the Mayer–Vietoris short exact sequence and enters only at the small-chains comparison, and compact support became a named lemma where the textbook offers only a shrug.

Several layers are natural Mathlib upstreaming candidates, in dependency order: the generator normal forms for the singular chain complex (`Idx/gen/gen_d/gen_map`); the prism operator and Theorems 3.2 and 3.3, which are stated against Mathlib declarations already; the affine-chain subdivision theory, which is self-contained combinatorics; the small-chains theorem and the Mayer–Vietoris sequence; and the augmentation apparatus with the sphere detectors. No part of this development has been merged into Mathlib, and no claim of that kind is made here; adaptation to Mathlib’s evolving simplicial API, and generalization from $\mathbb{Z}$ to arbitrary coefficients where the

Paper statement	Lean declaration(s)	Module
Thm. 3.1	<code>prism_chain_homotopy_succ, ..._zero</code>	<code>SingularPrism</code>
Thm. 3.2	<code>homotopic_maps_induce_same_homology</code>	<code>SingularPrism</code>
Thm. 3.3	<code>homotopyEquiv_homology_iso,</code> <code>isIso_homology_map_of_homotopyEquiv</code>	<code>SingularPrism</code>
Thm. 4.1	<code>pairδ, pair_les_exact$_{1/2/3}$</code>	<code>SingularPair</code>
$H_n(X, X) = 0$	<code>relative_homology_id_isZero</code>	<code>SingularPair</code>
Thm. 5.1	<code>abnd_comp_atее, abnd_comp_atееIter</code>	<code>SingularSubdivision</code>
Thm. 5.2	<code>asubIter_support_bound,</code> <code>exists_asubIter_small</code>	<code>SingularSubdivision</code>
Thm. 5.3	<code>exists_sdOpIter_small</code>	<code>SingularSubdivision</code>
Thm. 6.1	<code>smallι_isIso_homologyMap,</code> <code>smallChainsHomologyIso</code>	<code>SingularMayerVietoris</code>
MV chain SES	<code>mvSES_shortExact</code>	<code>SingularMayerVietoris</code>
Thm. 6.2	<code>mvδ, mv_exact$_{1/2/3}$, mvSum_epi_zero</code>	<code>SingularMayerVietoris</code>
Thm. 7.1	<code>isIso_augH_of_pathConnected</code>	<code>SingularSphere</code>
Thm. 7.2	<code>isZero_homology_of_contractible</code>	<code>SingularSphere</code>
Suspension iso	<code>suspensionIso</code>	<code>SingularSphereGeometry</code>
Thm. 7.3	<code>sphere_homology_vanish</code>	<code>SingularSphereGeometry</code>
Thm. 7.4	<code>h1_s1_ne_zero, sphere_top_ne_zero</code>	<code>SingularSphereGeometry</code>
Thm. 7.5	<code>spheres_not_homotopyEquivalent,</code> <code>sphere_dim_eq_of_homotopyEquiv</code>	<code>SingularSphereGeometry</code>
Thm. 8.1	<code>unknotComplementH1_ne_zero</code>	<code>UnknotComplementRetract</code>
Thm. 8.2	<code>not_detects_zero, not_detects_one</code>	<code>LinkingVanishingLowDim</code>
Thm. 8.3	<code>isZero_h1_complement_of_embedding,</code> <code>not_detects_of_arcAcyclic,</code> <code>forces_D3_of_arcAcyclic</code>	<code>LinkingVanishingHighDim</code>
Thm. 8.4	<code>arcComplementsAcyclic</code>	<code>ArcComplementAcyclic</code>
Detector	<code>DetectsNontrivialLinking</code>	<code>PublicSpine</code>
Thm. 9.1 ($\Leftarrow$)	<code>detectsNontrivialLinking_three</code>	<code>PublicSpine</code>
Thm. 9.1 ($\Rightarrow$)	<code>forces_D3, target_D3</code>	<code>PublicSpineLinkingClosure</code>
$H_1(S^1) \cong \mathbb{Z}$ (imported)	<code>circleH1ZIsoInt_holds</code>	<code>CircleWindingChain</code>

Table 2: Theorem index. All declarations are in namespace `IndisputableMonolith.Foundation`, in the sub-namespace matching the module name.

proofs allow it, is the main anticipated work. Beyond upstreaming, two mathematical continuations are within reach of the existing scaffolding: the isomorphism $H_n(S^n) \cong \mathbb{Z}$ by induction from the winding-chain base case through `suspensionIso`, and the extension of Theorem 8.4 to all degrees and to disk complements, which would move the development from a corridor of Alexander-type control toward duality proper.

Acknowledgments

The author thanks the Lean and Mathlib communities for the foundations this work builds on.

References

- [1] J. Washburn. *The Unique Dimension in Which an Embedded Circle Links: A Machine-Checked Proof that Complement H_1 Forces $D = 3$* . In preparation, 2026.

- [2] A. Hatcher. *Algebraic Topology*. Cambridge University Press, 2002.
- [3] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.
- [4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, LNCS 12699, pages 625–635. Springer, 2021.
- [5] J. Harrison. A HOL Light formalization of singular homology. Presentation at Hales60: From the Fundamental Lemma to Discrete Geometry, to Formal Verification, 2018.
- [6] L. C. Paulson. Porting libraries of mathematics between proof assistants. Blog post and the HOL-Homology session of the Isabelle distribution, 2022.
- [7] G. Brunerie. *On the homotopy groups of spheres in homotopy type theory*. PhD thesis, Université Nice Sophia Antipolis, 2016.
- [8] U. Buchholtz and K. Hou (Favonia). Cellular cohomology in homotopy type theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2018)*, pages 521–529. ACM, 2018.
- [9] A. Ljungström and A. Mörtberg. Formalizing $\pi_4(S^3) \cong \mathbb{Z}/2\mathbb{Z}$ and computing a Brunerie number in cubical Agda. In *Proceedings of the 38th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2023)*. IEEE, 2023.
- [10] A. Topaz et al. Formalization of cohomology theories. BIRS workshop report 23w5124, Banff International Research Station, 2023.